Edge Computing

run the small, latency-sensitive bit close to the user

Watch the 4-min video

In one line: CDNs now run your code in hundreds of locations near users; use it for lightweight, latency-sensitive logic that doesn't need central, strongly consistent data (routing, token checks, A/B assignment, rewrites, bot filtering, picking cached variants), and keep heavy or transactional work in your region.

1 400 ms in Sydney

Users in Sydney wait about 400 ms for the homepage because every request travels to a server in Virginia and back. The page is the same for everyone except the language and an experiment variant, and that small difference is why it isn't cached. Physics sets the floor: you can't make Virginia closer, so move the logic, and the cache, to Sydney.

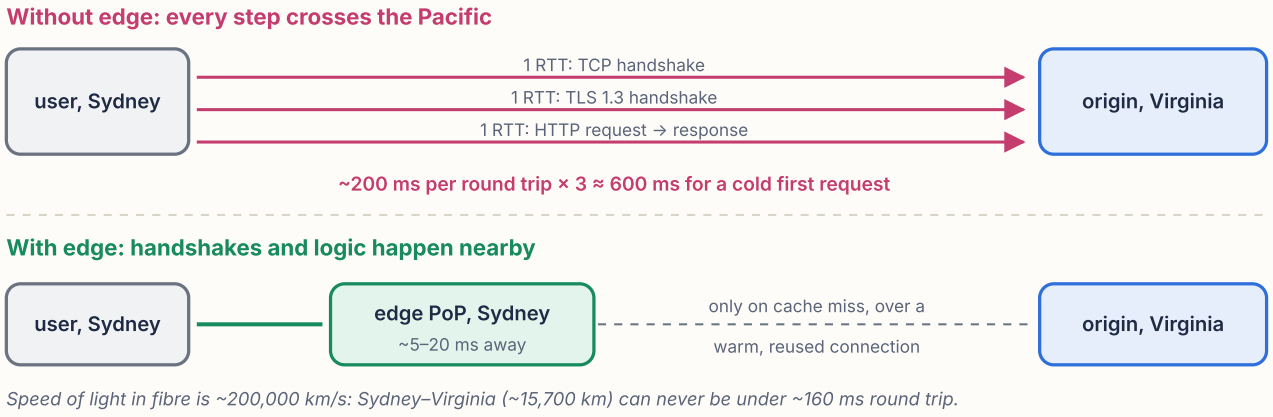


Figure 1. Why distance hurts: new connections need several round trips. An edge location terminates TCP and TLS nearby and reuses warm connections to the origin.

2 Code at the CDN edge

CDNs already cache static files in hundreds of points of presence (PoPs). **Edge compute** lets you run your own code there. The platforms differ a lot:

Platform	Runtime	Shape and limits (check current docs)
Cloudflare Workers	V8 isolates: JS/TS, WebAssembly	runs in every PoP; CPU-time limits per request (10 ms on the free plan, much higher on paid); KV, Durable Objects, D1 for state
CloudFront Functions	lightweight JavaScript	viewer request/response only; sub-millisecond budget, tiny memory, no network calls; very cheap
Lambda@Edge	Node.js, Python	runs in regional edge caches; all four hooks; network calls allowed; 5 s timeout for viewer hooks, 30 s for origin hooks
Fastly Compute	WebAssembly (Rust, Go, JS...)	fast-starting Wasm instances per request; edge KV and config stores
Azure Front Door rules engine	declarative rules, not code	match conditions → rewrite, redirect, header and cache-key actions

Isolates (Workers) start in about a millisecond because many tenants share one process, each in its own sandboxed V8 context; this is why cold starts are a smaller concern than for regional serverless functions, though large bundles and heavy libraries still slow them.

3 Edge functions run on every request

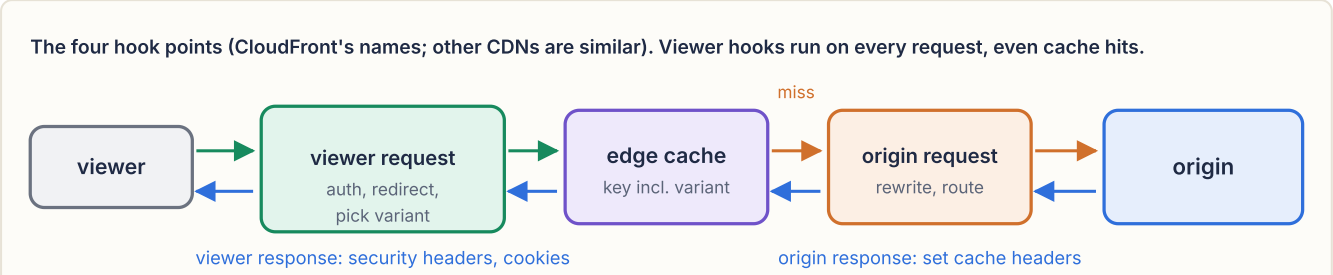


Figure 2. Where edge code hooks into a request. Logic before the cache can change what is looked up; logic after a miss can change what is fetched.

Running **before** the cache, a function can rewrite URLs (`/` → `/en-AU/home`), redirect by country, validate a token and reject with 401, block bots, add headers, and decide **which cached object** to serve. Each of those saves a full trip to the origin, or prevents a bad request from ever reaching it.

4 Personalisation at cache speed

The naive approach to personalisation marks the whole page uncacheable, so every request goes to Virginia. Instead, find the few dimensions that actually vary (language, experiment group, logged-in vs anonymous, device class) and make each combination a **cached variant**: the edge function reads a cookie or header, normalises it to a small set of values, and adds it to the **cache key**. With 3 languages × 2 variants, there are 6 cached copies, all served at edge speed. Keep the number of combinations small, or the hit rate collapses; never put a user ID in the cache key. Per-user fragments (the name in the header) can be filled client-side from a small API call.

5 The edge has limits

- **Tight CPU and memory budgets**, from under a millisecond (CloudFront Functions) to tens of milliseconds or more per request; restricted APIs (no arbitrary sockets or file systems on some platforms).

- **State is eventually consistent**: edge key-value stores replicate writes globally, often taking seconds up to a minute to appear everywhere. Fine for config and feature flags, not for balances. Strongly consistent options exist (Cloudflare Durable Objects put one object in one location), but then requests travel to it.
- **Your database is in one region**. An edge function that queries it on every request just moves the slow round trip, and can make it worse by opening new connections. Edge wins only when the work completes at the edge, or the origin call is a rare cache miss.
- **Operational differences**: debugging and observability are harder, deploys are global within seconds (a bug is global too), and vendor-specific APIs mean lock-in.

6 What suits the edge



Figure 3. Rule of thumb: if it needs only the request plus slowly changing config, it can run at the edge.

Token checks are a good example of the boundary: an edge function can **verify a JWT's signature and expiry** using cached public keys (JWKS) with no network call, rejecting forged or expired tokens instantly. It can't check whether a token was revoked a second ago without asking a central service; keep token lifetimes short or accept that window.

WORKED EXAMPLE: A/B TEST ASSIGNMENT AT THE EDGE

A visitor arrives in Sydney without an `exp` cookie. The edge function takes a visitor ID (from an existing anonymous cookie, or a new random one), computes `hash(visitor_id + experiment_name) mod 100`, and assigns **A** if the result is below 50, else **B**. Hashing makes assignment **deterministic and sticky** without storing anything, and including the experiment name keeps experiments independent. It sets `exp=homepage:B` on the response and looks up the cache with the variant in the key (`/home|en-AU|B`). Both variants are cached at the Sydney PoP, so the page arrives in tens of milliseconds, with **no flicker** (unlike client-side A/B tools that swap content after load) and no trip to Virginia. Exposure events go to analytics asynchronously, and changing the split is a config update pushed to the edge.

COMMON MISTAKES

- Calling the central database from the edge on every request.
- Cache keys with too many variants (or a user ID), so nothing is ever a hit.
- Storing strongly consistent data (stock, balances) in an eventually consistent edge KV.
- Heavy npm dependencies that blow the CPU budget or cold-start time.

7 Key terms

PoP: point of presence: a CDN edge location

Edge function: your code running in CDN locations, per request

Isolate: lightweight sandbox sharing one runtime process; very fast start

Cache key: the values that identify a cached object

Edge KV: globally replicated, eventually consistent key-value store

Origin: your own backend that the CDN fetches from

8 Interview questions

Q1. What workloads suit edge compute? BASIC

Lightweight, latency-sensitive logic that doesn't need central, strongly consistent data: routing and redirects, token checks, A/B assignment, header and URL rewrites, bot filtering, and choosing cached variants for personalisation.

Q2. What are its limits? BASIC

Tight CPU and memory budgets, restricted APIs, only eventually consistent edge storage, and a central database that is still far away.

Q3. How does the edge personalise without losing caching? BASIC

It reduces the request to a few variant values (language, experiment group), adds them to the cache key, and serves the matching cached variant.

Q4. Your edge function queries the regional database on each request and got slower. Why? SCENARIO

Each request still crosses to the region, now possibly on new connections from many PoPs; cache the data at the edge or keep the logic at the origin.

Q5. How would you assign A/B variants consistently without a database? SCENARIO

Hash a stable visitor ID with the experiment name, take it mod 100 against the split, and store the result in a cookie; the same visitor always gets the same variant.

Q6. Can you do authentication at the edge? TRICKY

You can verify signed tokens (signature, expiry, audience) with cached public keys; revocation checks and session lookups still need a central store or short token lifetimes.

Q7. Why do isolates start faster than regional serverless functions? TRICKY

They reuse an already running runtime process and only create a new sandboxed context, rather than booting a container or micro-VM.

Next: SD-V-020 · Platform Engineering backendroadmap.com